

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

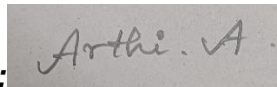
No. of Questions: 5

Annexure A Algorithm Design Challenge

Code of Conduct:

I ARTHI A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given Romania Map Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Q1. Greedy Best-First Search – Romania Map

Find a route from Arad (source) to Bucharest (goal) in the Romania road-map graph using Greedy Best-First Search (GBFS). The algorithm should use a heuristic to guide the search toward Bucharest.

Initial state: Arad. Goal state: Bucharest. States: Romanian cities. Actions: move along connected roads. Assumptions: the map is known, roads are traversable, and the heuristic values are available. Constraints: only connected roads may be used; repeated states should be avoided. Expected outcome: a valid route to Bucharest.

Greedy Best-First Search, an informed search method. It selects the frontier node with the smallest heuristic $h(n)$.

GBFS is appropriate because the Romania problem provides a useful geographic heuristic: straight-line distance (SLD) to Bucharest. It is more goal-directed than BFS/DFS and often expands fewer nodes. However, unlike A*, it ignores the cost already travelled, so it is not guaranteed to produce the shortest route.

Nodes represent cities and edges represent roads. A useful trace is Arad → Sibiu → Făgăraș → Bucharest. The search frontier is ordered by increasing $h(n)$.

Not Applicable. This is a graph-search problem rather than a CSP.

$h(n)$ = straight-line distance from city n to Bucharest. Example values: $h(\text{Arad})=366$, $h(\text{Sibiu})=253$, $h(\text{Făgăraș})=176$, $h(\text{Bucharest})=0$. SLD is admissible because straight-line distance does not exceed actual road distance, but GBFS itself is not optimal.

1. Put the source in OPEN. 2. Set CLOSED empty. 3. Select the OPEN node with minimum $h(n)$. 4. If it is the goal, reconstruct and return the path. 5. Otherwise expand its unvisited neighbours. 6. Add them to OPEN with parent pointers. 7. Repeat until the goal is found or OPEN becomes empty.

Leave the right-side space for the Draw.io flowchart / Romania search graph.

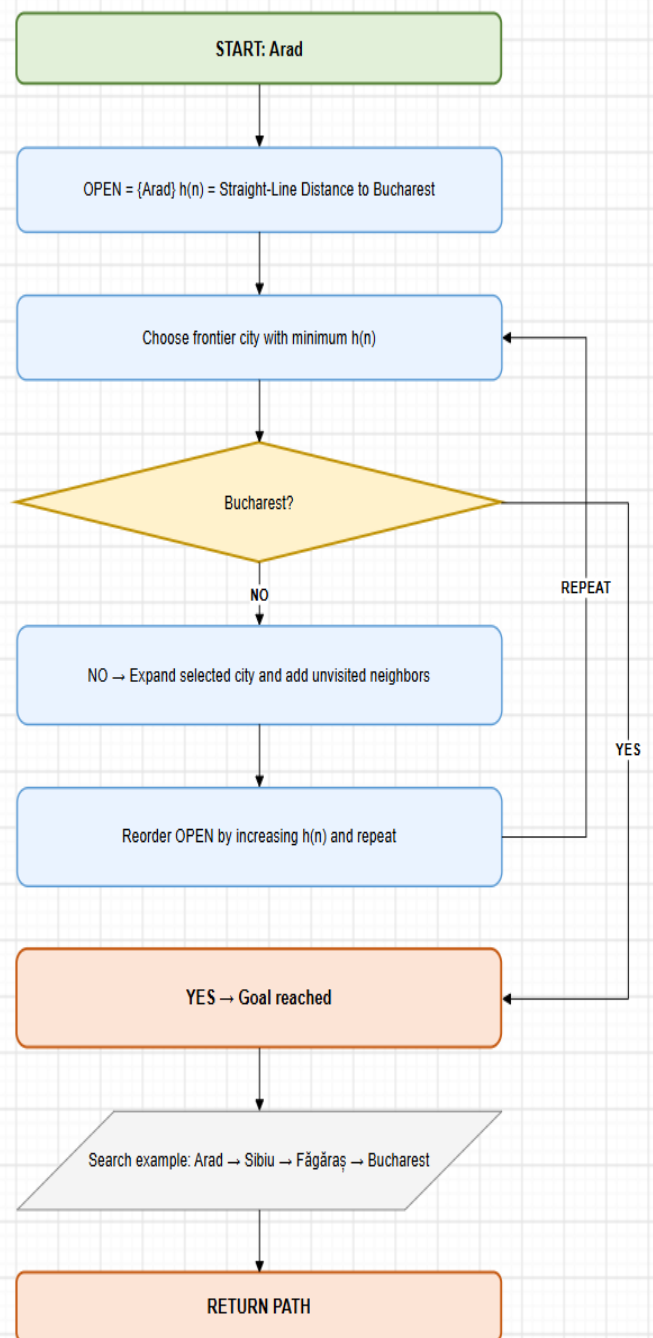
Start at Arad (366). Frontier includes Sibiu (253), Timisoara (329), Zerind (374). Choose Sibiu because 253 is smallest. From Sibiu, Făgăraș has $h=176$ and is selected. From Făgăraș, Bucharest has $h=0$ and is selected. Path obtained: Arad → Sibiu → Făgăraș → Bucharest.

Worst-case time: $O(b^m)$, where b is branching factor and m is maximum depth. Worst-case space: $O(b^m)$, because frontier and explored information may grow exponentially.

Advantages: simple, fast in many cases, heuristic-guided, and goal-directed. Limitations: not guaranteed optimal or complete in every formulation; can be misled by a poor heuristic and may choose a long route.

GBFS is effective for quickly reaching a destination when a strong heuristic is available.

FLOWCHART / DIAGRAM SPACE



Q2. A* Search – Delivery Robot

A delivery robot must travel from a warehouse/source S to a customer/goal G through a weighted road network. The objective is to find a minimum-cost route.

Initial state: warehouse S. Goal: customer G. States: road junctions. Actions: travel along connected roads. Assumptions: edge costs are non-negative and the map is known. Constraints: only valid roads can be used and the route should minimize total cost.

A* Search is selected because it combines the actual cost from the source with an estimate of the remaining cost.

A* is preferable to GBFS when optimality is required because it uses $f(n)=g(n)+h(n)$. With an admissible heuristic, A* tree search is optimal; with a consistent heuristic, graph-search A* is complete and optimal under standard non-negative-cost assumptions.

Road junctions are states and weighted roads are transitions. Each node stores $g(n)$, $h(n)$, $f(n)$, and a parent pointer.

Not Applicable.

$h(n)$ is a lower-bound estimate of remaining travel cost. Straight-line distance is appropriate when road cost is proportional to distance. Admissibility requires $h(n) \leq$ true remaining cost. Consistency requires $h(n) \leq c(n,n') + h(n')$.

1. $OPEN \leftarrow \{S\}$; $CLOSED \leftarrow \emptyset$; $g(S)=0$. 2. Compute $f(S)=g(S)+h(S)$. 3. Remove the node with minimum f . 4. If it is G, reconstruct the path. 5. For each neighbour calculate tentative g . 6. If the new cost is lower, update parent, g and f and place/update the node in OPEN. 7. Repeat.

Leave the right-side space for the Draw.io A* flowchart.

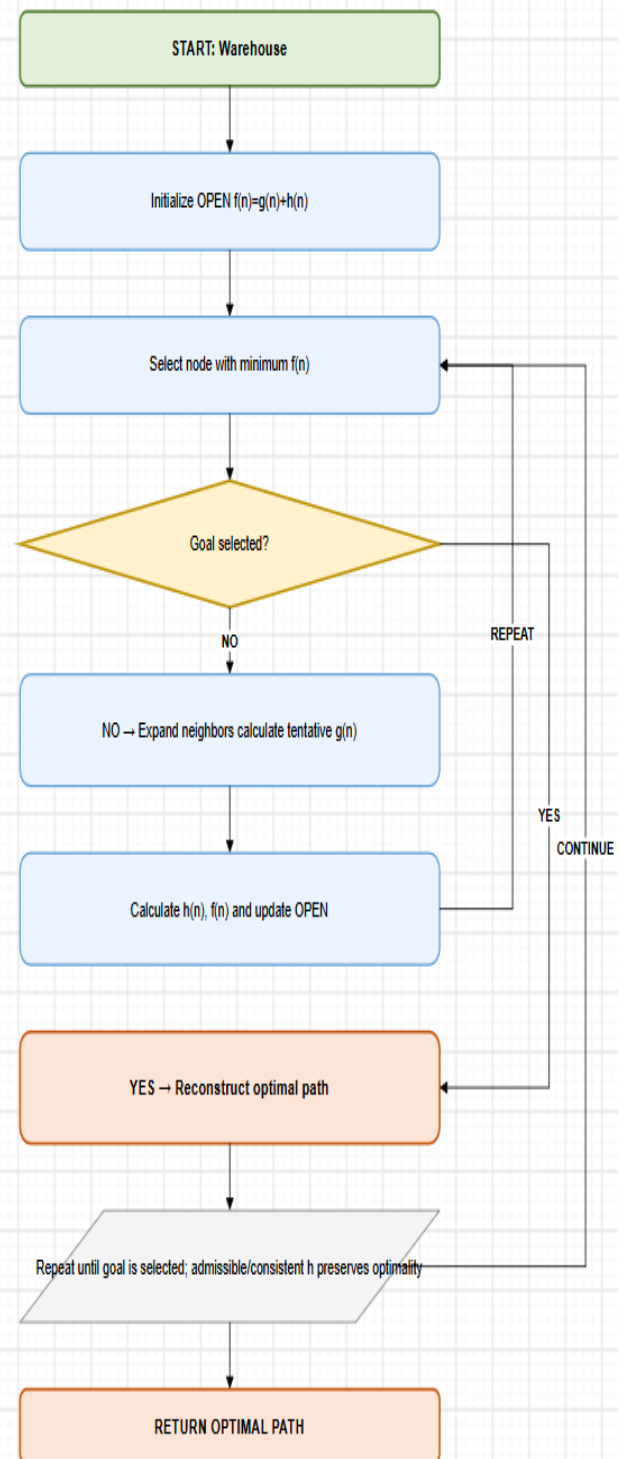
Illustrative weighted network: $S \rightarrow A$ cost 2, $A \rightarrow E$ cost 4, $E \rightarrow F$ cost 4, $F \rightarrow G$ cost 2. Example heuristic values: $h(S)=8$, $h(A)=7$, $h(E)=3$, $h(F)=1$, $h(G)=0$. Therefore S: $g=0, h=8, f=8$; A: $g=2, h=7, f=9$; E: $g=6, h=3, f=9$; F: $g=10, h=1, f=11$; G: $g=12, h=0, f=12$. The selected route has total cost 12 in this illustrative network.

Worst-case time and space are $O(b^d)$ for a general search tree, where b is branching factor and d is solution depth. A good heuristic can substantially reduce practical expansions.

Advantages: complete under standard conditions, optimal with suitable heuristic, balances past and future cost. Limitations: high memory usage and performance depends strongly on heuristic quality.

A* is the appropriate algorithm when the delivery robot must find an efficient route while guaranteeing minimum cost under the required heuristic conditions.

FLOWCHART / DIAGRAM SPACE



Q3. Examination Timetabling – CSP, MRV and Forward Checking

Create an examination timetable so that subjects with common students are not scheduled at the same time while all exams receive valid slots.

Initial state: no exam assigned. Goal: every exam assigned exactly one legal slot. Assumptions: a fixed number of time slots is available. Constraints: conflicting exams cannot share a slot; additional room/capacity restrictions may apply.

Constraint Satisfaction Problem solved with backtracking, enhanced by Minimum Remaining Values (MRV) and Forward Checking.

CSP directly models exams as variables, slots as domains, and conflicts as constraints. MRV chooses the most constrained variable first, while Forward Checking removes values that are no longer possible. This detects dead ends early and reduces unnecessary search compared with blind backtracking.

Use a constraint graph: each exam is a vertex and an edge connects two exams that conflict. Assigning a time slot is equivalent to coloring the graph.

Variables: E1,E2,E3,E4,E5,E6. Domain for each: {T1,T2,T3,T4}. Example constraints: E1≠E2, E1≠E3, E2≠E4, E3≠E5, E4≠E6, E2≠E5. If room capacity is included, each slot must also satisfy total room capacity.

Not Applicable as an informed-search heuristic. MRV is a variable-ordering heuristic: choose the unassigned variable with the smallest remaining domain. Degree heuristic can break MRV ties.

1. If all variables are assigned, return solution. 2. Select variable using MRV. 3. Try each legal value. 4. Assign the value. 5. Forward-check all neighbouring unassigned variables. 6. If any domain becomes empty, undo and try another value. 7. Otherwise recurse. 8. If all values fail, backtrack.

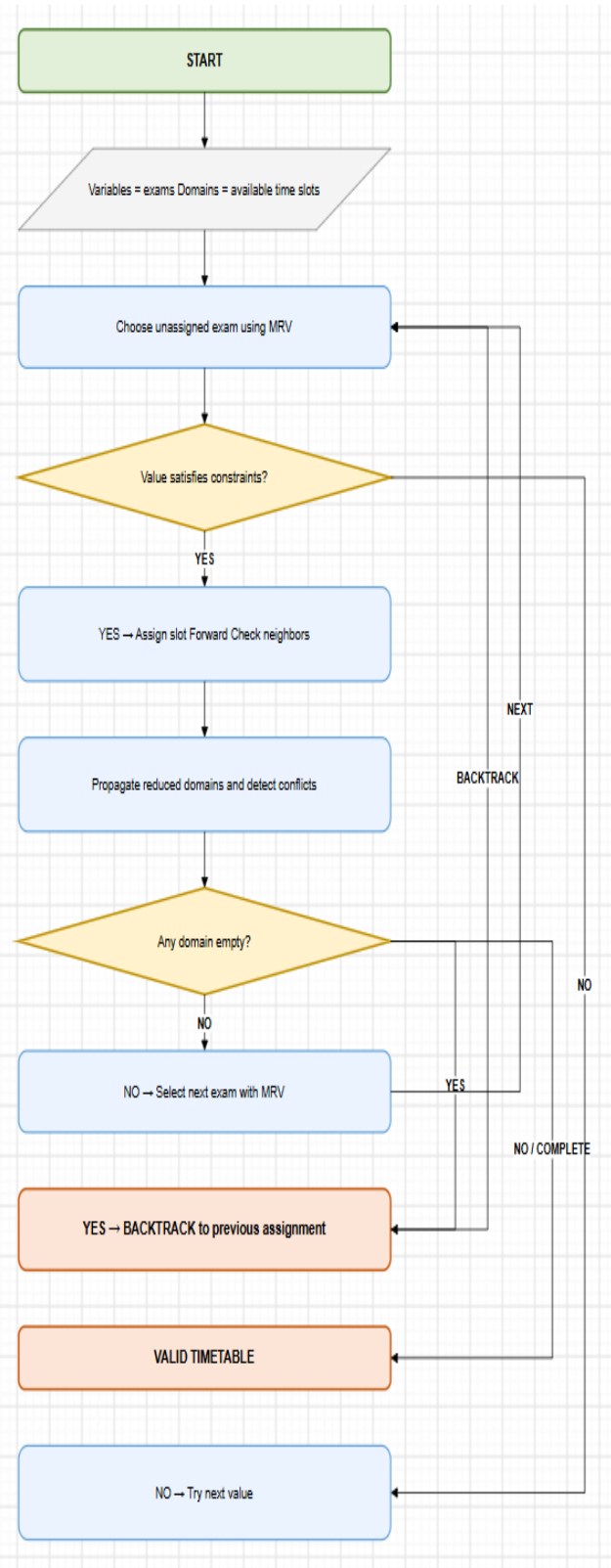
Leave the right-side space for the Draw.io CSP flowchart.

Initially each exam has {T1,T2,T3,T4}. Suppose E1 is selected by MRV and assigned T1. Forward Checking removes T1 from E2 and E3. Assign E2=T2; remove T2 from E4 and E5. Assign E3=T2; remove T2 from E5. Continue until all exams have legal slots. If any domain becomes empty, backtrack to the most recent decision.

With n variables and maximum domain size d , worst-case time is $O(d^n)$. Space is $O(n)$ recursion depth plus domain storage. MRV and Forward Checking do not change the worst-case exponential bound but greatly reduce practical search.

Advantages: natural representation of conflicts, early failure detection, systematic search, and improved efficiency with MRV/Forward Checking. Limitations: worst-case exponential complexity and performance depends on constraint structure.

FLOWCHART / DIAGRAM SPACE



Q4. Minimax and Alpha-Beta Pruning

Determine the best move for the MAX player in a deterministic two-player game by assuming that both players make optimal decisions.

Initial state: root game position. Goal: choose the move with the highest guaranteed utility. MAX maximizes the score; MIN minimizes it. Assumptions: zero-sum, deterministic, perfect-information game. Constraints: legal moves and terminal utility values define the game tree.

Minimax is used to select the optimal move. Alpha-Beta Pruning is added to reduce unnecessary evaluation.

Minimax provides the correct decision against a rational opponent.

Represent the supplied game as a tree: root MAX node → MIN nodes → MAX nodes → terminal utility leaves. The exact terminal values from the given game tree should be written beside the corresponding leaves.

Not Applicable. Game states and legal moves define the search space.

If the full tree is available, a heuristic is not required because terminal utilities can be evaluated. For depth-limited search, use an evaluation function estimating board quality from MAX's perspective.

MINIMAX: If terminal, return utility. If MAX, return maximum child value. If MIN, return minimum child value. ALPHA-BETA: start $\alpha = -\infty$, $\beta = +\infty$; at MAX update $\alpha = \max(\alpha, \text{value})$; at MIN update $\beta = \min(\beta, \text{value})$; if $\alpha \geq \beta$, prune remaining siblings.

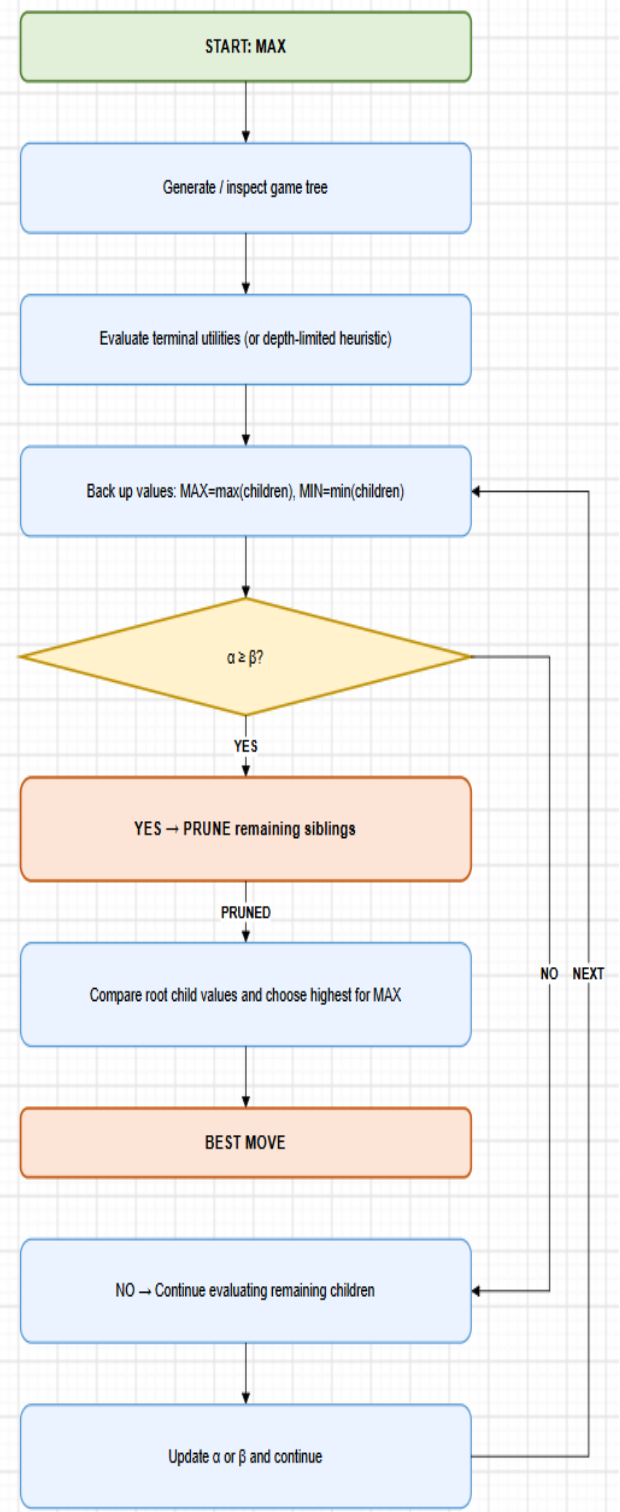
Leave the right-side space for the Draw.io Minimax/Alpha-Beta flowchart and the supplied game tree.

Evaluate terminal leaves from left to right. Propagate values upward using MAX and MIN rules. During Alpha-Beta evaluation, maintain α and β . When $\alpha \geq \beta$, mark all remaining siblings under that node as PRUNED. The root selects the child with the highest backed-up value. Because the actual game-tree leaf values were not included in the text of this assessment, the final numeric best move must be calculated directly from the supplied tree.

Minimax: $O(b^d)$ time and $O(bd)$ depth-first space. Alpha-Beta: worst case $O(b^d)$, but with excellent move ordering the effective complexity can approach $O(b^{d/2})$; space remains $O(bd)$ for depth-first implementation.

Advantages: rational optimal play, exact decision under assumptions, and substantial pruning with good move ordering. Limitations: exponential growth, dependence on tree depth, and Alpha-Beta efficiency depends on move ordering.

FLOWCHART / DIAGRAM SPACE



Q5. Online Search Agent – Dynamic Warehouse Robot

A warehouse robot must reach a target while obstacles may appear after planning. The robot therefore needs to sense, act, update its knowledge, and replan.

Initial state: robot's current cell and partially known map.
Goal: target shelf/cell. Assumptions: sensors can detect nearby obstacles and the robot can move safely.
Constraints: blocked cells cannot be entered and the environment can change during execution.

Online Search Agent using repeated A* on the robot's current knowledge of the environment. This combines informed search with continuous sensing and replanning.

Offline informed search assumes the environment remains as known. An online agent is better for dynamic environments because it can discover changes and modify its route. A* provides a strong path-planning component while online feedback handles uncertainty.

Grid cells are states; movement between adjacent cells forms edges. Known obstacles are blocked. Unknown cells become known through sensing. The current robot position changes after every action.

Not Applicable.

For four-direction unit-cost grid movement, $h(n) = |x - x_{goal}| + |y - y_{goal}|$ (Manhattan distance). It is admissible and consistent for that movement model.

1. Sense the environment. 2. Update the known map. 3. Run A* from current position to goal. 4. Execute only the next safe action. 5. Sense again. 6. If a new obstacle blocks the route, mark it blocked and replan. 7. Continue until the goal is reached.

Leave the right-side space for the Draw.io Online Search Agent flowchart.

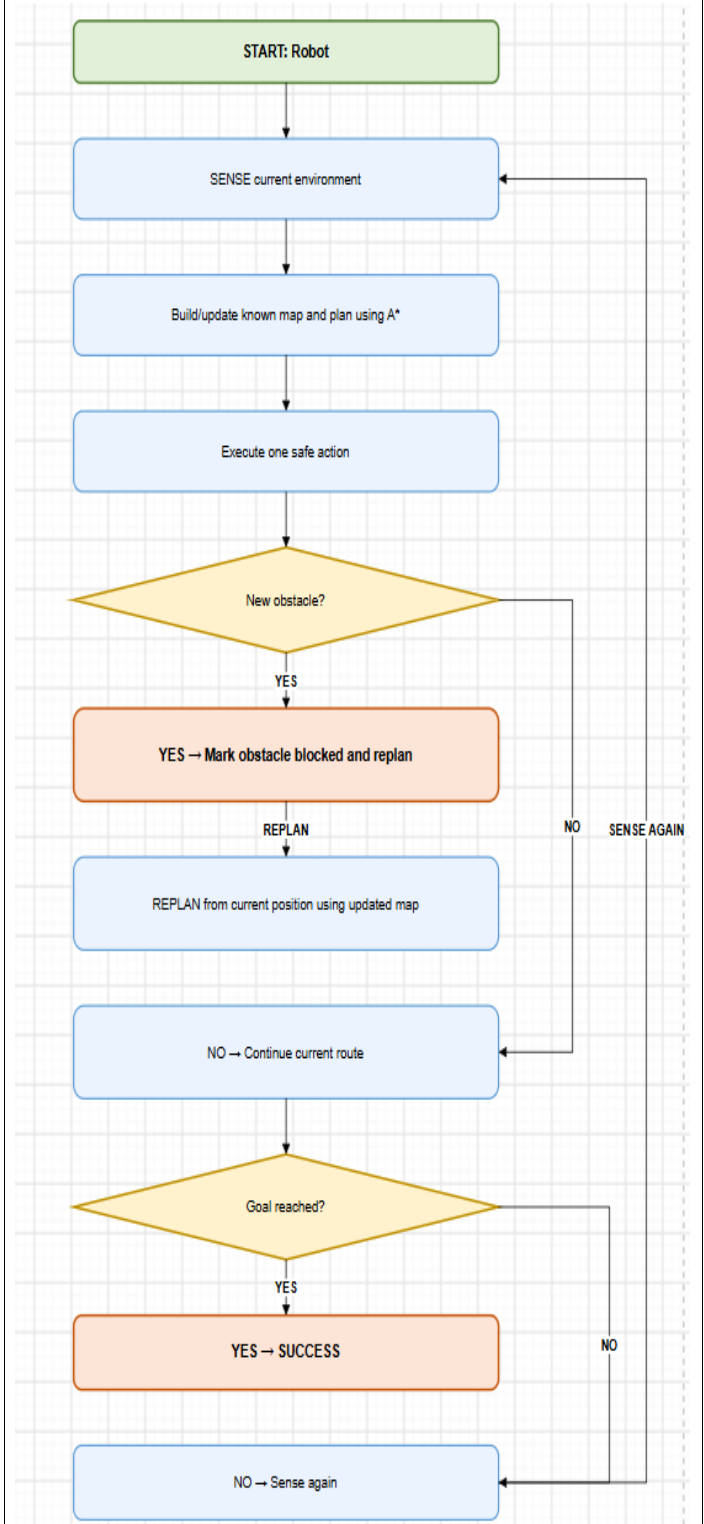
Robot senses a clear route and A* plans through cells $A \rightarrow B \rightarrow C \rightarrow D$. After moving to B, a new obstacle is detected at C. The robot marks C as blocked, updates its map, reruns A*, and selects an alternative route $B \rightarrow E \rightarrow F \rightarrow D$. This demonstrates adaptation rather than following a fixed plan.

Each A* replanning episode has worst-case $O(b^d)$ time and $O(b^d)$ space. Because replanning can happen multiple times, total cost depends on the number of environmental changes. Incremental algorithms can reduce repeated work.

Advantages: adapts to unexpected obstacles, uses current sensor information, safer in changing environments, and can retain informed-search benefits. Limitations: repeated replanning consumes computation and may be inefficient in extremely dynamic environments.

An online A* search agent is a practical solution for dynamic warehouse navigation.

FLOWCHART / DIAGRAM SPACE



Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.